DSA 8070 R Session 3: Multivariate Normal Distribution, Copula Models, and Nonparametric Density Estimation

Whitney Huang, Clemson University

Contents

Multivariate Normal Distribution

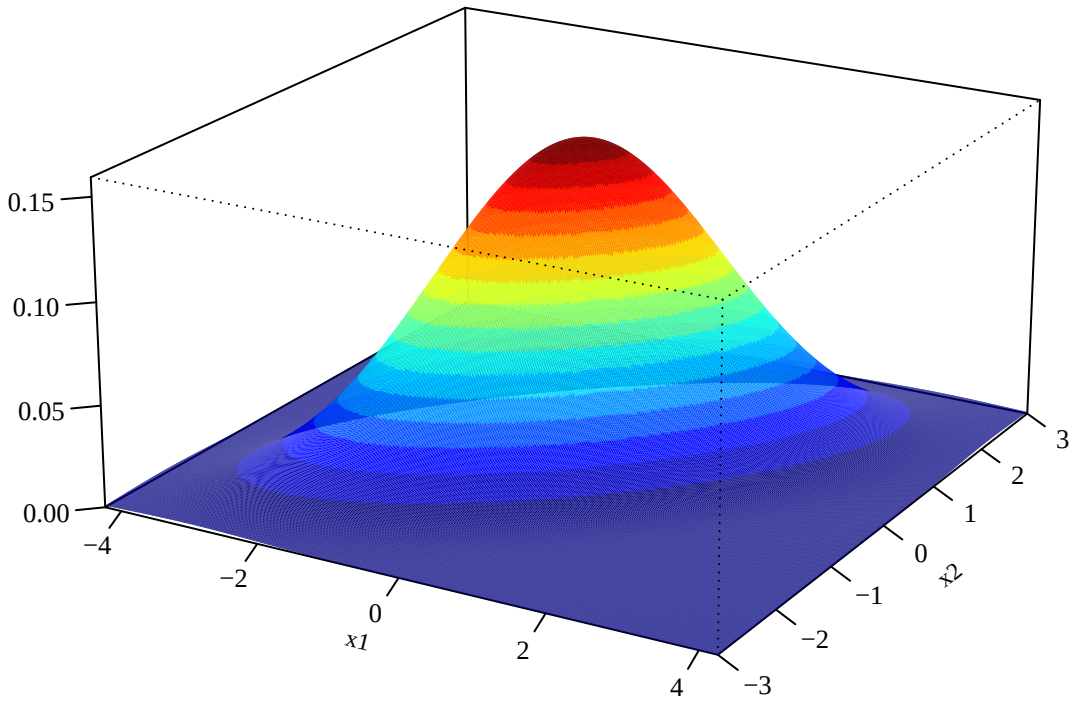
Bivariate normal density

If X_1 and X_2 jointly follows a bivariate normal distribution, then the probability density function takes the following form:

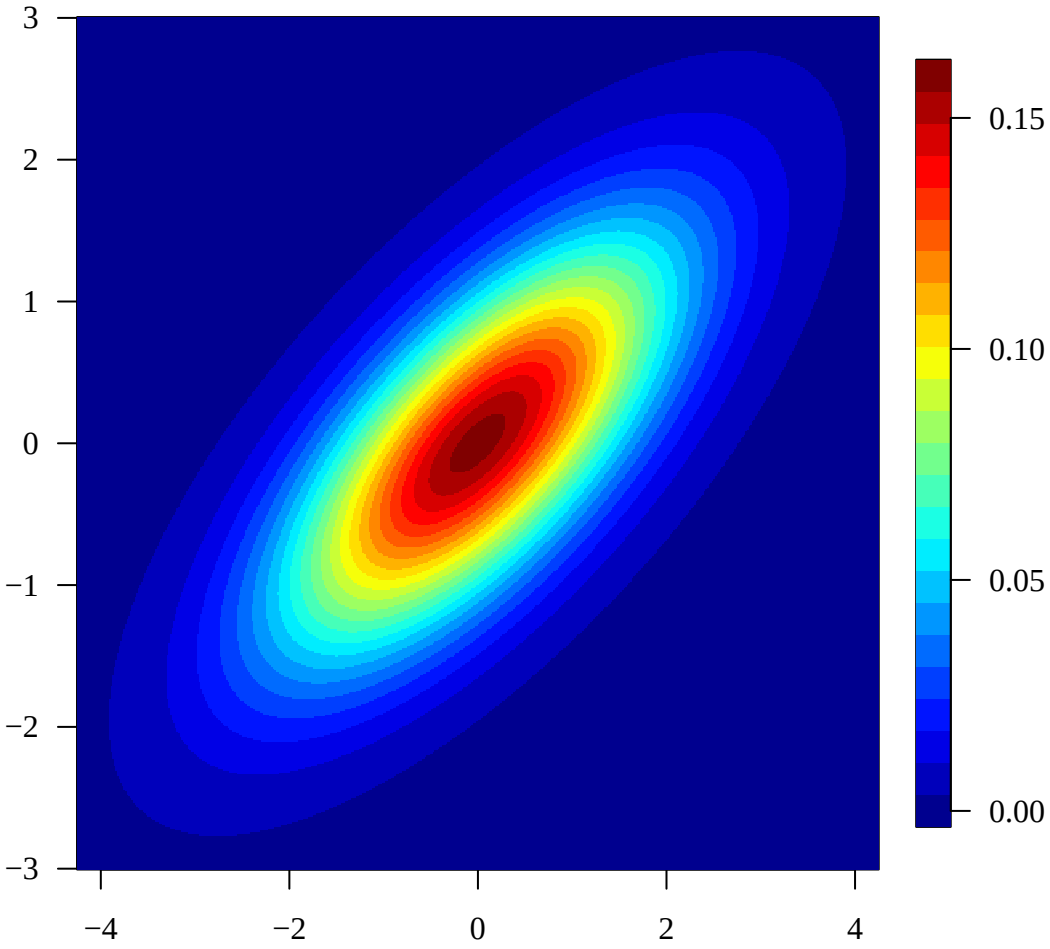
$$f(x_1, x_2) = \frac{1}{2\pi\sigma_1\sigma_2\sqrt{1-\rho^2}} \exp\left(-\frac{1}{2(1-\rho^2)}\left[\left(\frac{x_1-\mu_1}{\sigma_1}\right)^2 + \left(\frac{x_2-\mu_2}{\sigma_2}\right)^2 - 2\rho\frac{(x_1-\mu_1)(x_2-\mu_2)}{\sigma_1\sigma_2}\right]\right),$$

where $[\mu_1, \mu_2]^T$ is the mean vector, and $\begin{bmatrix}\sigma_{11}=\sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_{22}=\sigma_2^2\end{bmatrix}$ is the covariance matrix.

```r
## Specify parameters of the bivariate normal distribution
mu1 <- mu2 <- 0
sigma11 <- 2 # variance of X1
sigma22 <- 1 # variance of X2
sigma12 <- 1 # covariance between X1 and X2

# Construct the covariance matrix
Sigma <- matrix(c(sigma11, sigma12, sigma12, sigma22), 2)

## Create a grid of values for evaluating the density
# Use +/- 3 marginal standard deviations around each mean
x1 <- seq(mu1 - 3 * sqrt(sigma11), mu1 + 3 * sqrt(sigma11),
          length = 500)
x2 <- seq(mu2 - 3 * sqrt(sigma22), mu2 + 3 * sqrt(sigma22),
          length = 500)

# Evaluate the bivariate normal density over the grid
library(mvtnorm)
grids <- expand.grid(x1, x2)
out <- array(dmvnorm(grids, c(mu1, mu2), Sigma),
             dim = c(500, 500))

## Plot the bivariate normal density as a 3D surface
par(mar = c(2, 2, 0.8, 1.3), mgp = c(2.2, 1, 0),
    family = "serif")
library(GA)
persp3D(x1, x2, out, theta = 30, phi = 20, expand = 0.5,
        zlab = "", cex.axis = 0.8, cex.lab = 0.75)

## Plot the same density from above using a color map
# Colors represent the height of the probability density
library(fields)
```




```
image.plot(x1, x2, out, las = 1, col = tim.colors(24),
           legend.cex = 0.5, legend.mar = 7)
```

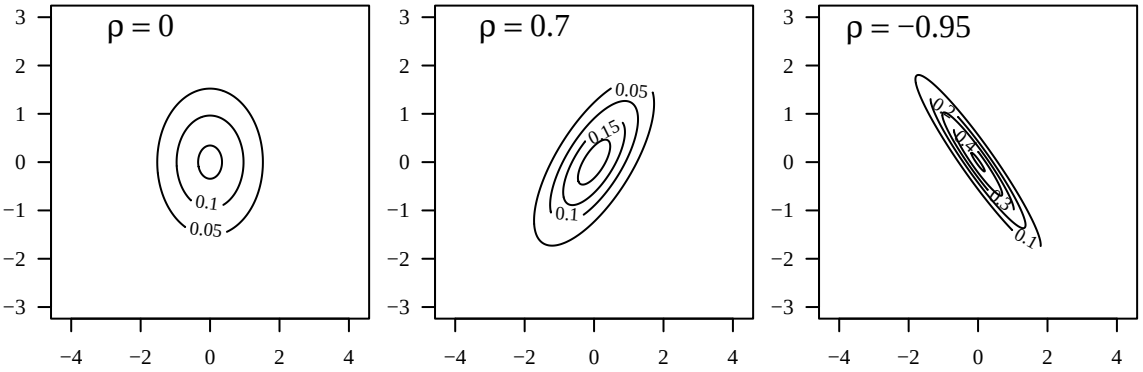


Bivariate normal with different ρ

To illustrate the effect of correlation on a bivariate normal distribution, we fix both marginal variances at 1 and vary the correlation ρ. When $\rho = 0$, the density contours are circular. As $|\rho|$ increases, the contours become more elongated, while the sign of ρ determines their orientation. Thus, the correlation controls both the strength and direction of the linear dependence.

```r
## Compare bivariate normal density contours under different correlations
par(mfrow = c(1, 3), mar = c(2, 2, 0.8, 0.6), family = "serif")

# Set the mean vector and three correlation values
mu <- c(0, 0)
rho <- c(0, 0.7, -0.95)

# Generate one contour plot for each correlation
for (i in 1:3){
  # Construct the covariance matrix with unit marginal variances
  Sigma <- matrix(c(1, rho[i], rho[i], 1), 2)
  # Evaluate the bivariate normal density over the plotting grid
  f <- array(dmvnorm(grids, mu, Sigma), dim = c(500, 500))
  # Plot equal-density contours
  contour(x1, x2, f, las = 1, nlevels = 5)
  # Display the corresponding correlation
```

```
  text(-2, 2.75, bquote(rho == .(rho[i])), cex = 1.5)
}
```



Multivariate normal QQplot

1. Calculate the Hotelling's squared statistics

$$d^2 = (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu)$$

   for each multivariate data point $\boldsymbol{x}_i, \quad i = 1, \cdots, n.$

2. Sort $\{d_i\}_{i=1}^n$ in increasing order
3. Compute the theoretical quantiles under multivariate normal condition, which are the chi-squared quantiles with degrees of freedom equal to its dimension p (i.e., the number of variables)
4. Create a scatterplot with both empirical and theoretical quantiles.
5. Add a reference line to assess to straightness.

```
# Extract the first three variables for the 50 versicolor observations
versicolor <- iris[51:100, 1:3]

# Estimate the mean vector and covariance matrix
(mu <- apply(versicolor, 2, mean))
```

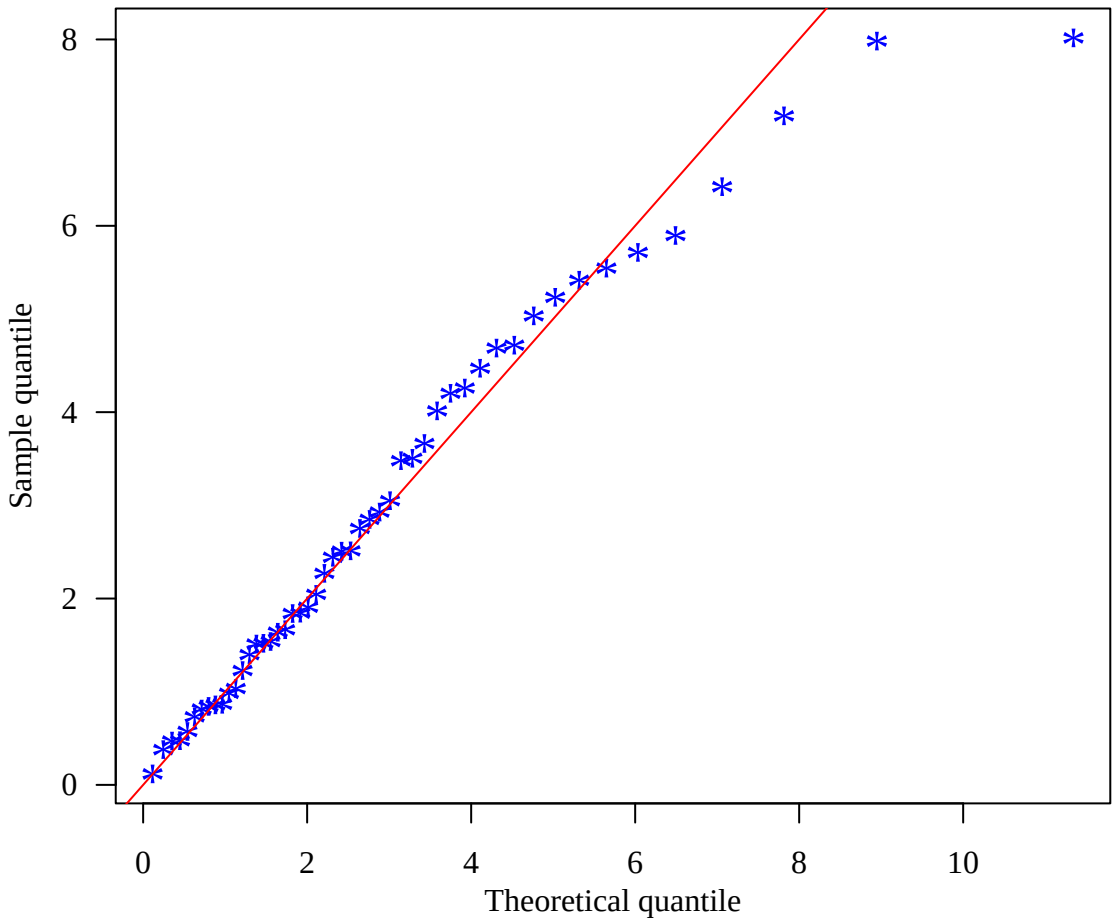
```
## Sepal.Length  Sepal.Width Petal.Length
##        5.936        2.770        4.260
```

```
(Sigma <- cov(versicolor))
```

```
##              Sepal.Length Sepal.Width Petal.Length
## Sepal.Length   0.26643265  0.08518367   0.18289796
## Sepal.Width    0.08518367  0.09846939   0.08265306
## Petal.Length   0.18289796  0.08265306   0.22081633
```

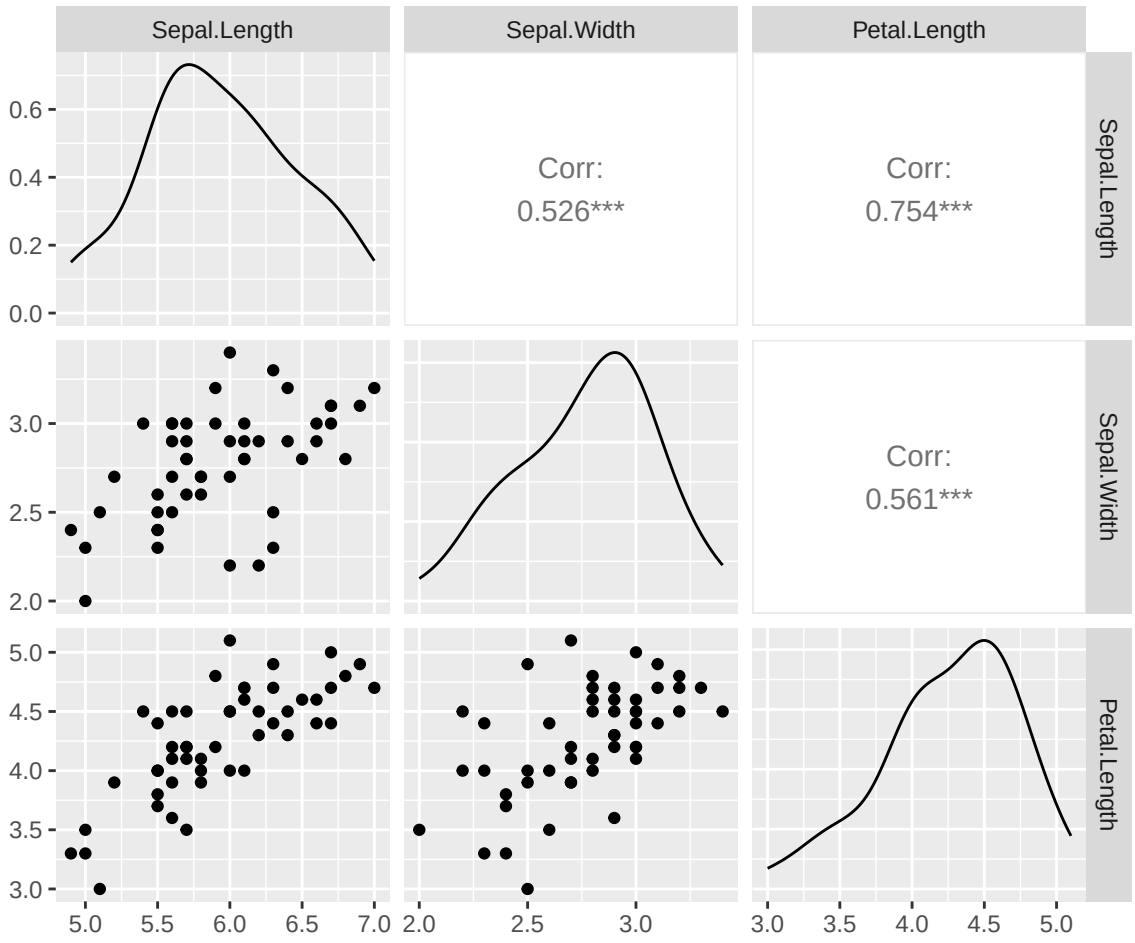
```
# Compute squared Mahalanobis distances (empirical quantiles)
d.square <- apply(versicolor, 1, function(x) t(x - mu) %*% solve(Sigma) %*% (x - mu))

# Get the sample size
```

```r
n <- dim(versicolor)[1]

# Compute theoretical quantiles from a chi-square distribution with p = 3 degrees of freedom
chiquant <- qchisq((1:n - 0.5) / n, 3)

# Construct the chi-square Q-Q plot
par(las = 1, mgp = c(2, 1, 0), mar = c(3.5, 3.5, 0.8, 0.6),
    family = "serif")
plot(chiquant, sort(d.square), pch = "*", col = "blue",
     xlab = "Theoretical quantile", ylab = "Sample quantile", cex = 1.6)

# Add the 45-degree reference line
abline(0, 1, col = "red")
```



The χ^2 Q–Q plot shows some departure from the pattern expected under multivariate normality. To investigate the source of this departure, we return to the data and examine the marginal distributions and pairwise relationships among the variables. A scatterplot matrix can help identify features such as marginal non-normality, unusual observations, or non-elliptical dependence that may explain the observed departure.

```r
# Examine the marginal distributions and pairwise relationships
# to identify possible sources of the departure
library(GGally)
ggpairs(versicolor, progress = F)
```



Probability contour

We now visualize how the eigenvalues and eigenvectors of the covariance matrix determine the geometry of a bivariate normal probability ellipse. For a $100(1-\alpha)\%$ probability ellipse, the principal directions are given by the eigenvectors of $\mathbf{\Sigma}$, while the corresponding semi-axis lengths are

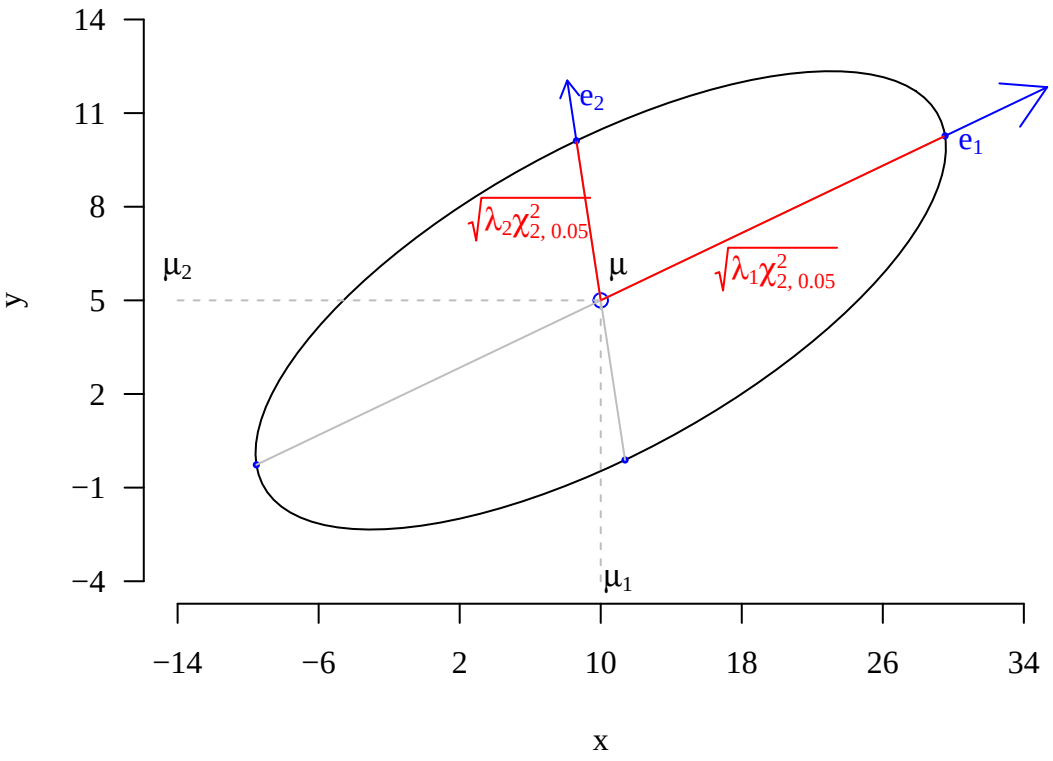
$$\sqrt{\chi^2_{2,1-\alpha}\lambda_1} \quad \text{and} \quad \sqrt{\chi^2_{2,1-\alpha}\lambda_2}.$$

The following code constructs the 95% probability ellipse and explicitly displays its center, principal directions, and semi-axis lengths.

```
## Specify the mean vector, covariance matrix, and plotting range
mu <- c(10, 5)
Sigma <- matrix(c(64, 16, 16, 9), 2)

# Compute marginal standard deviations and use +/- 3 SDs as plotting limits
sd1 <- sqrt(diag(Sigma)[1]); sd2 <- sqrt(diag(Sigma)[2])
xr <- c(mu[1] - 3 * sd1, mu[1] + 3 * sd1)
yr <- c(mu[2] - 3 * sd2, mu[2] + 3 * sd2)

## Perform the spectral decomposition of the covariance matrix
# Eigenvalues determine the lengths of the principal axes,
# while eigenvectors determine their directions
```

```r
spetralDecomp <- eigen(Sigma)
lambda1 <- spetralDecomp$values[1]
lambda2 <- spetralDecomp$values[2]
e1 <- spetralDecomp$vectors[, 1]
e2 <- spetralDecomp$vectors[, 2]

## Construct the 95% probability ellipse
# The boundary corresponds to squared Mahalanobis distance = chi-square(2, 0.95)
c <- sqrt(qchisq(0.95, 2))

# Convert covariance to correlation for ellipse()
library(ellipse)
cor <- Sigma[1, 2] / (sd1 * sd2)

# Draw the probability ellipse centered at mu
par(family = "serif", las = 1)
plot(ellipse(cor, scale = sqrt(diag(Sigma)), centre = mu), type = 'l',
xlim = xr, ylim = yr, las = 1, bty = "n", yaxt = "n", xaxt = "n", las = 1)

# Mark the mean vector, which is the center of the ellipse
points(mu[1], mu[2], col = "blue")

# Add axes using one-marginal-SD increments
axis(1, at = seq(mu[1] - 3 * sd1, mu[1] + 3 * sd1, sd1))
axis(2, at = seq(mu[2] - 3 * sd2, mu[2] + 3 * sd2, sd2))

## Calculate the major and minor semi-axis vectors
# Semi-axis length in direction e_j is c * sqrt(lambda_j)
majorX <- c * sqrt(lambda1) * -e1[1]
majorY <- c * sqrt(lambda1) * -e1[2]
minorX <- c * sqrt(lambda2) * -e2[1]
minorY <- c * sqrt(lambda2) * -e2[2]

## Mark the endpoints of the two principal axes
points(mu[1] + majorX, mu[2] + majorY, pch = 16, cex = 0.5, col = "blue")
points(mu[1] - majorX, mu[2] - majorY, pch = 16, cex = 0.5, col = "blue")
points(mu[1] + minorX, mu[2] + minorY, pch = 16, cex = 0.5, col = "blue")
points(mu[1] - minorX, mu[2] - minorY, pch = 16, cex = 0.5, col = "blue")

## Draw and label the major principal axis
# Its direction is e1 and its semi-axis length is c * sqrt(lambda1)
segments(mu[1] + majorX, mu[2] + majorY,
         mu[1] - majorX, mu[2] - majorY, col = "gray")
segments(mu[1], mu[2], mu[1] + majorX, mu[2] + majorY, col = "red")
text(20, 6, expression(sqrt(lambda[1]*chi["2, 0.05"]^2)), col = "red")

## Draw and label the minor principal axis
# Its direction is e2 and its semi-axis length is c * sqrt(lambda2)
segments(mu[1] + minorX, mu[2] + minorY,
         mu[1] - minorX, mu[2] - minorY, col = "gray")
segments(mu[1], mu[2], mu[1] + minorX, mu[2] + minorY, col = "red")
text(6, 7.6, expression(sqrt(lambda[2]*chi["2, 0.05"]^2)), col = "red")
```

```r
## Extend arrows from the ellipse to indicate the eigenvector directions
arrows(mu[1] + majorX, mu[2] + majorY,
       mu[1] + majorX + (-6) * e1[1], mu[2] + majorY + (-6) * e1[2],
       col = "blue")
arrows(mu[1] + minorX, mu[2] + minorY, mu[1] + minorX + (-2) * e2[1],
       mu[2] + minorY + (-2) * e2[2], length = 0.1, col = "blue")

## Add reference lines and labels for the mean vector
segments(10, -4, 10, 5, col = "gray", lty = 2)
text(11, -4, expression(mu["1"]))
segments(-14, 5, 10, 5, col = "gray", lty = 2)
text(-14, 6, expression(mu["2"]))
text(11, 6, expression(bolditalic(mu)))

## Label the two eigenvector directions
text(31, 10, expression(e["1"]), col = "blue")
text(9.5, 11.4, expression(e["2"]), col = "blue")
```



Wechsler Adult Intelligence Scale Example

The `wechsler` data contain measurements from the **Wechsler Adult Intelligence Scale (WAIS)**, a widely used test of cognitive ability. Here we consider four test components—Information, Similarities, Arithmetic, and Picture Completion—measured on 37 individuals. We use these data to illustrate a practical assessment of multivariate normality by examining the covariance structure, squared Mahalanobis distances, and pairwise relationships among the variables.

```r
## Read the Wechsler Adult Intelligence Scale data
# The first column is an identifier, so it is excluded from the analysis
```

```r
data <- read.table("wechsler.txt")

## Estimate the mean vector and covariance matrix
(mu <- apply(data[, -1], 2, mean))
```

```
##        V2        V3        V4        V5
## 12.567568  9.567568 11.486486  7.972973
```

```r
(Sigma <- cov(data[, -1]))
```

```
##           V2         V3        V4        V5
## V2 11.474474  9.0855856  6.382883 2.0713213
## V3  9.085586 12.0855856  5.938438 0.5435435
## V4  6.382883  5.9384384 11.090090 1.7912913
## V5  2.071321  0.5435435  1.791291 3.6936937
```

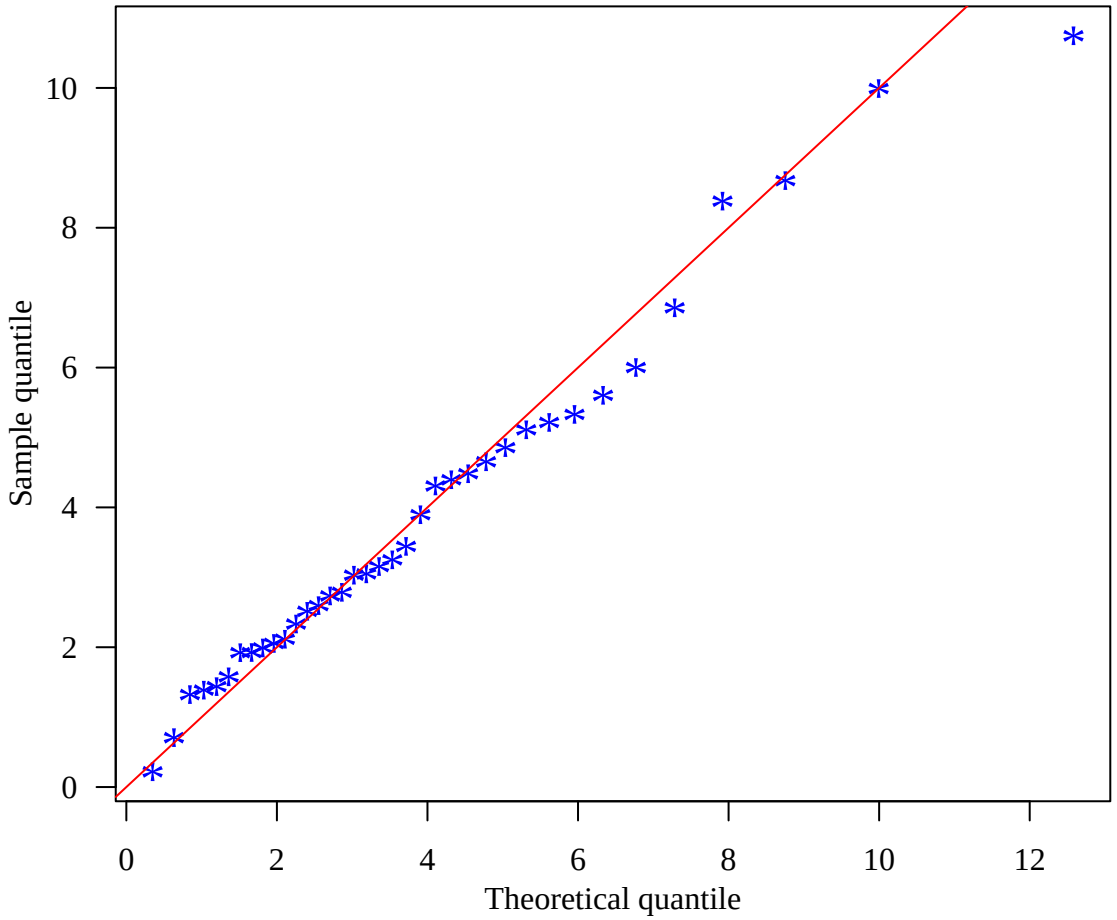
```r
## Examine the covariance geometry through spectral decomposition
# Eigenvalues describe variation along the principal directions,
# while eigenvectors give the corresponding directions
out <- eigen(Sigma)
out$values
```

```
## [1] 26.245278  6.255366  3.931553  1.911647
```

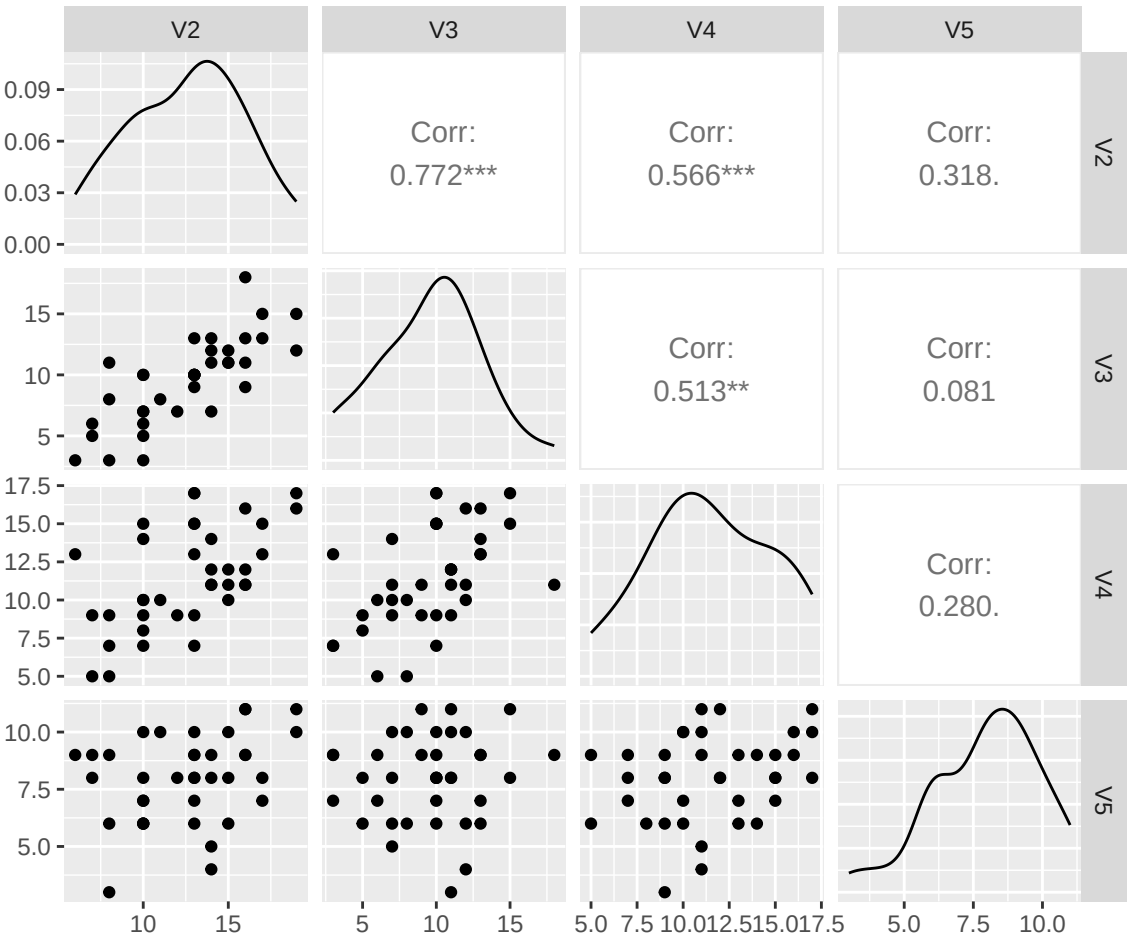
```r
out$vectors
```

```
##            [,1]       [,2]       [,3]        [,4]
## [1,] -0.6057467 -0.2176473  0.4605028  0.61125912
## [2,] -0.6047618 -0.4958117 -0.3196759 -0.53501516
## [3,] -0.5051337  0.7946452 -0.3349263  0.03468877
## [4,] -0.1103360  0.2744802  0.7573433 -0.58216643
```

```r
## Assess multivariate normality using squared Mahalanobis distances
# Under multivariate normality, these distances approximately follow chi-square(p)
d.square <- apply(data[, -1], 1, function(x) t(x - mu) %*% solve(Sigma) %*% (x - mu))

# Determine the sample size n and dimension p
n <- dim(data)[1]; p <- dim(data[, - 1])[2]

# Compute theoretical quantiles from the chi-square distribution with p degrees of freedom
chiquant <- qchisq((1:n - 0.5) / n, p)

## Construct the chi-square Q-Q plot
# Approximate linearity supports the multivariate normal assumption;
# substantial departures may indicate non-normality or unusual observations
par(las = 1, mgp = c(2, 1, 0), mar = c(3.5, 3.5, 0.8, 0.6),
    family = "serif")
plot(chiquant, sort(d.square), pch = "*", col = "blue",
    xlab = "Theoretical quantile", ylab = "Sample quantile", cex = 1.6)

# Add the 45-degree reference line
abline(0, 1, col = "red")
```




```r
## Further investigate the marginal and pairwise distributions
# This can help explain departures observed in the chi-square Q-Q plot
library(GGally)
ggpairs(data[, -1], progress = F)
```



Copula Models

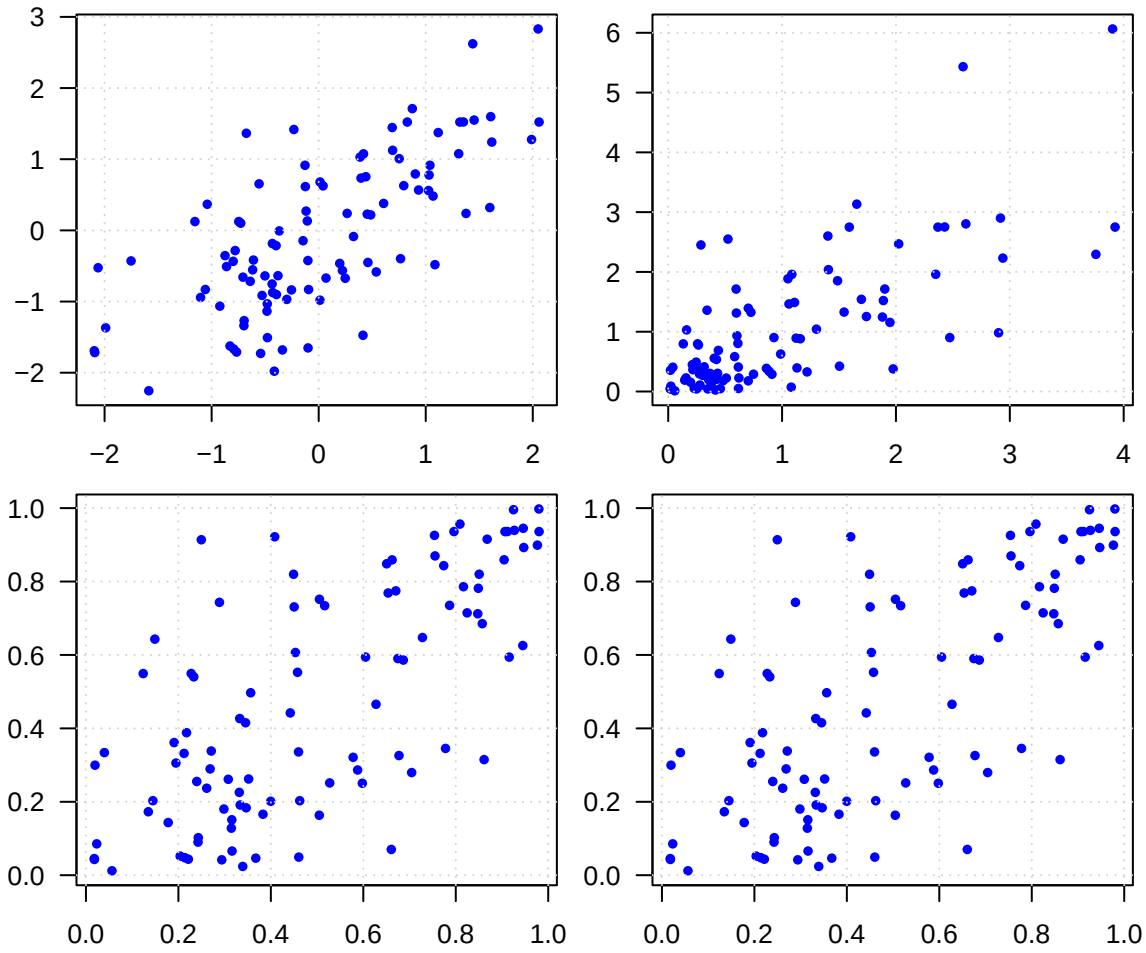
An Illustration of Bivariate Gaussian Copula

We now illustrate the basic idea of a copula through simulation. We first generate two correlated standard normal variables with correlation $\rho = 0.7$. Applying the standard normal CDF Φ to each variable transforms the marginal distributions to Uniform$(0, 1)$, while retaining the dependence structure on the probability scale. These uniform variables can then be transformed to other marginal distributions using their inverse CDFs. This illustrates the key idea of a copula: the marginal distributions can be changed separately from the dependence structure.

```
## Generate data from a bivariate normal distribution
# Both margins are standard normal with correlation rho = 0.7
mu <- c(0, 0)
Sigma <- matrix(c(1, 0.7, 0.7, 1), 2)

library(MASS)
normal <- mvrnorm(n = 100, mu, Sigma)

## Set up a 2 x 2 display for comparing the transformations
par(las = 1, mgp = c(2, 1, 0), mfrow = c(2, 2), mar = c(2, 2.4, 0.8, 0.6),
    fmaily = "serif")
```

```
## Warning in par(las = 1, mgp = c(2, 1, 0), mfrow = c(2, 2), mar = c(2, 2.4, :
## "fmaily" is not a graphical parameter
```

```r
# Plot the original bivariate normal observations
plot(normal, pch = 16, col = "blue", cex = 0.8, xlab = "", ylab = "")
grid()

## Apply the probability integral transform to each margin
# Since each margin is N(0,1), Phi(X_j) follows Uniform(0,1)
U <- apply(normal, 2, pnorm)

# Transform both uniform margins to exponential distributions
# The marginal distributions change, while the Gaussian copula is retained
plot(qexp(U[, 1]), qexp(U[, 2]), pch = 16, col = "blue", cex = 0.8, xlab = "", ylab = "")
grid()

# Plot the variables on the Uniform(0,1) probability scale
# This representation isolates the copula from the original margins
plot(U[, 1], U[, 2], pch = 16, col = "blue", cex = 0.8, xlab = "", ylab = "")
grid()

# Repeat the copula-scale plot
plot(U[, 1], U[, 2], pch = 16, col = "blue", cex = 0.8, xlab = "", ylab = "")
grid()
```

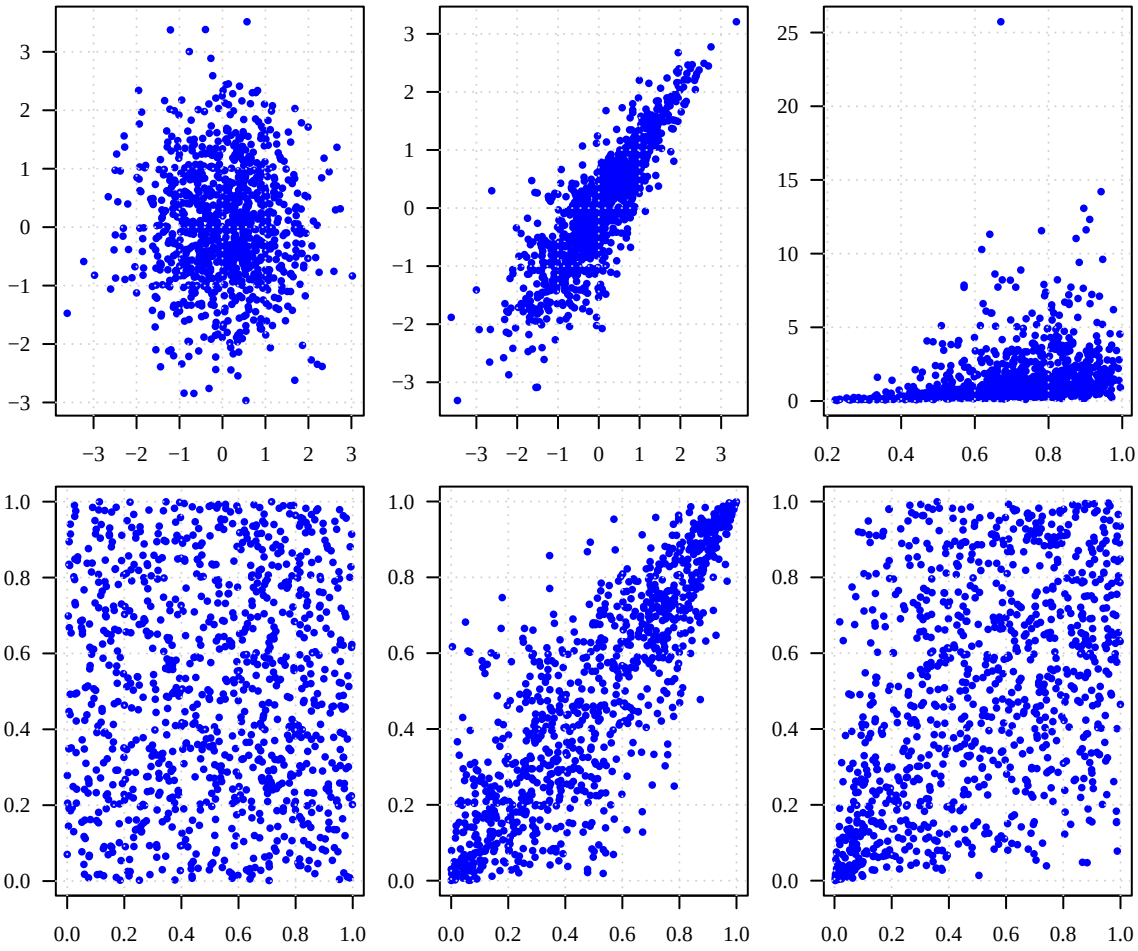


More Examples

- Left: normal + Gaussian copula ($\rho = 0$)
- Middle: normal + Gumbel copula
- Right: Beta + Log-normal + Clayton copula

```r
## Compare different copula models and marginal distributions
library(VineCopula)

# Arrange the plots in two rows:
# top row = observations on their original marginal scales
# bottom row = the corresponding copulas on the Uniform(0,1) scale
par(las = 1, mgp = c(2, 1, 0), mfrow = c(2, 3),
    mar = c(2, 2.4, 0.8, 0.6), family = "serif")

## Case 1: Independent standard normal margins
case1 <- cbind(rnorm(1000), rnorm(1000))
plot(case1, pch = 16, col = "blue", cex = 0.8, xlab = "", ylab = "")
grid()

## Case 2: Gumbel copula with standard normal margins
# The Gumbel copula produces asymmetric dependence with stronger upper-tail association
GumCop <- BiCop(family = 4, par = 3); UGum <- BiCopSim(1000, GumCop)
```

```r
# Transform the uniform copula variables to standard normal margins
plot(qnorm(UGum[, 1]), qnorm(UGum[, 2]), pch = 16, col = "blue", cex = 0.8,
     xlab = "", ylab = "")
grid()

## Case 3: Clayton copula with Beta and lognormal margins
# The Clayton copula produces asymmetric dependence with stronger lower-tail association
ClayCop <- BiCop(family = 3, par = 0.95); UClay <- BiCopSim(1000, ClayCop)

# Transform the first margin to Beta(5,2) and the second to lognormal
plot(qbeta(UClay[, 1], 5, 2), qlnorm(UClay[, 2]), pch = 16, col = "blue",
     cex = 0.8, xlab = "", ylab = "")
grid()

## Display the corresponding dependence structures on the copula scale

# Case 1: Independence copula obtained using the probability integral transform
U <- apply(case1, 2, pnorm)
plot(U[, 1], U[, 2], pch = 16, col = "blue", cex = 0.8, xlab = "", ylab = "")
grid()

# Case 2: Gumbel copula on the Uniform(0,1) scale
plot(UGum[, 1], UGum[, 2], pch = 16, col = "blue", cex = 0.8, xlab = "", ylab = "")
grid()

# Case 3: Clayton copula on the Uniform(0,1) scale
plot(UClay[, 1], UClay[, 2], pch = 16, col = "blue", cex = 0.8, xlab = "", ylab = "")
grid()
```



Real Data Example

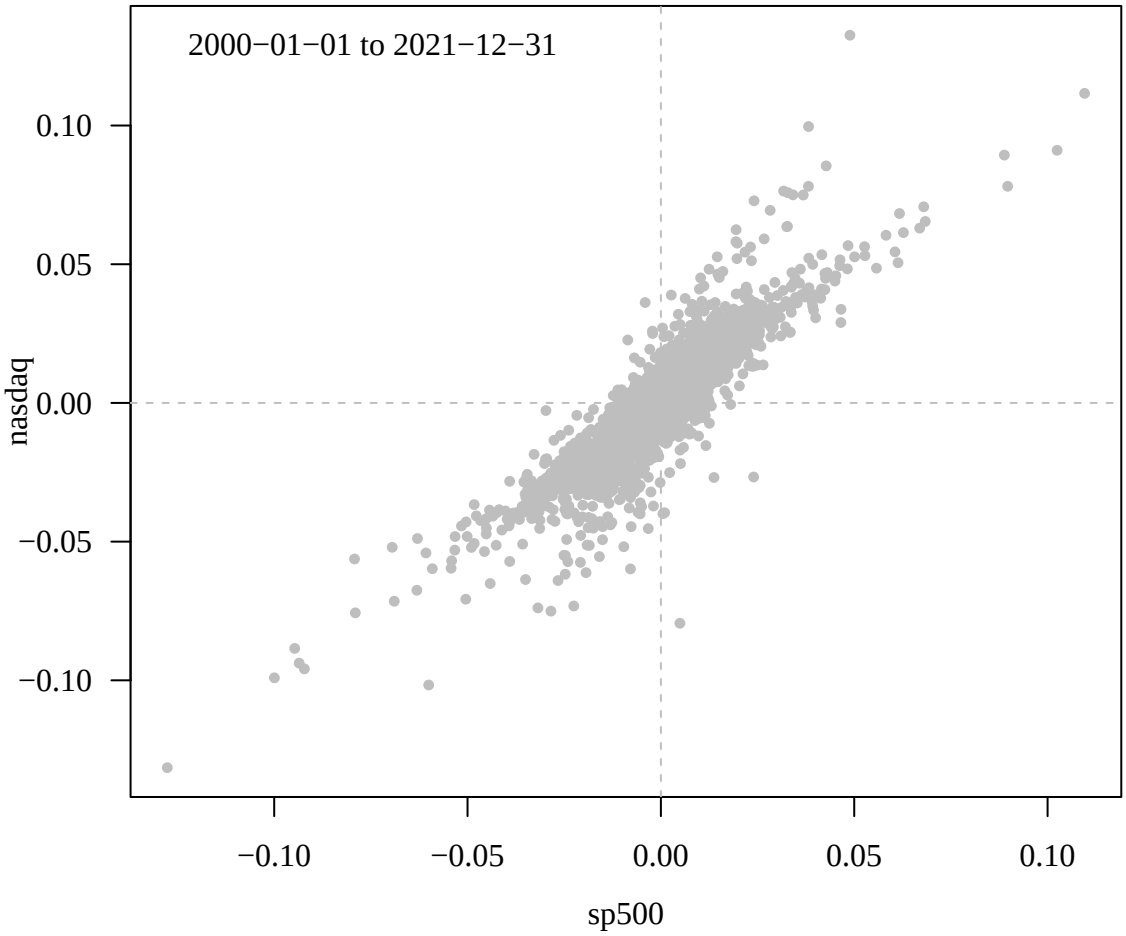
Here we illustrate how to use a copula to model the bivariate joint distribution of S&P 500 and Nasdaq (log) returns, where a normal distribution is assumed for each variable, while a Student-t copula is used to capture the dependence structure.

```
library(quantmod)

# Download historical daily data for the S&P 500 and NASDAQ Composite Index
# over the period from January 2000 through December 2021
getSymbols(c("^GSPC", "^IXIC"), from = "2000-01-01", to = "2021-12-31")
```

```
## [1] "GSPC" "IXIC"
```

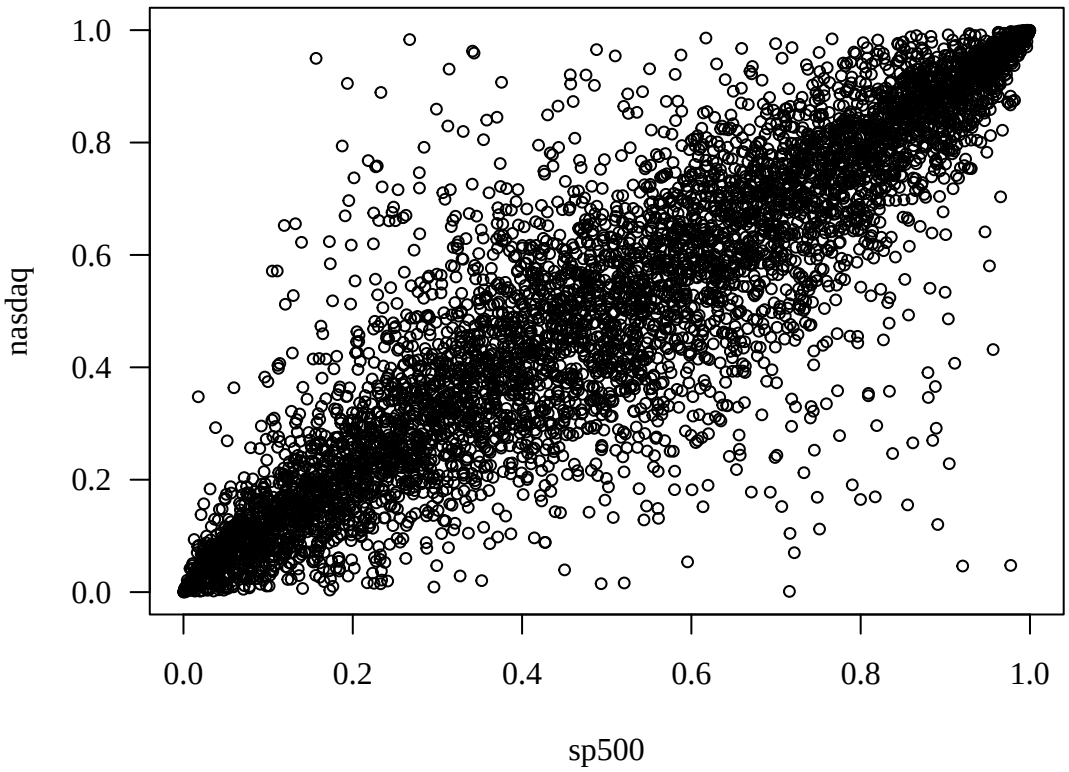
```
# Extract the daily closing prices for the two market indices
sp500 <- as.numeric(GSPC[, "GSPC.Close"])
nasdaq <- as.numeric(IXIC[, "IXIC.Close"])

# Combine the two closing-price series into a bivariate data set
data <- cbind(sp500, nasdaq)

# Calculate daily log returns: log(P_t) - log(P_{t-1})
```

```r
# Returns are used instead of prices to study daily market movements and dependence
returns <- diff(log(data), lag = 1)

## Plot S&P 500 returns against NASDAQ returns
# The scatterplot reveals the dependence between daily movements of the two indices
par(las = 1, mar = c(3.6, 3.6, 0.8, 0.6), mgp = c(2.5, 1, 0),
    family = "serif")
plot(returns, pch = 16, col = "gray", cex = 0.75)

# Add reference lines corresponding to zero return for each index
abline(v = 0, lty = 2, col = "gray")
abline(h = 0, lty = 2, col = "gray")

# Indicate the time period represented by the observations
legend("topleft", legend = "2000-01-01 to 2021-12-31", bty = "n")
```



Let's fit a copula model

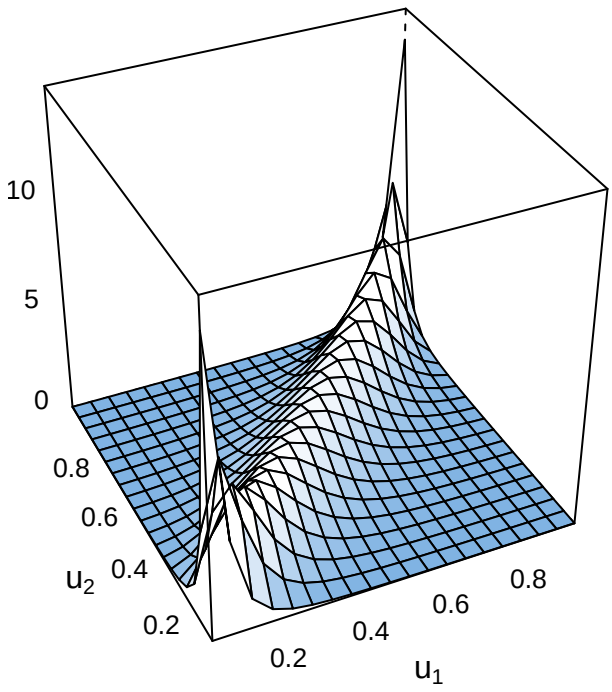
```r
## Transform the marginal returns to the Uniform(0,1) scale
# Use empirical ranks to obtain pseudo-observations for copula modeling
u <- apply(returns, 2, function(x) rank(x) / (length(x) + 1))
par(las = 1, family = "serif")
plot(u, cex = 0.75)
```




```
## Select an appropriate bivariate copula family
# Compare candidate copula families and select the preferred model
selectedCopula <- BiCopSelect(u[, 1], u[, 2], familyset = NA)
selectedCopula
```

```
## Bivariate copula: t (par = 0.92, par2 = 3.45, tau = 0.74)
```

```
## Fit the selected copula model
# Here family = 2 specifies a Student-t copula
copula_fit <- BiCopEst(u[, 1], u[, 2], family = 2)

## Visualize the fitted copula model
plot(copula_fit)
```



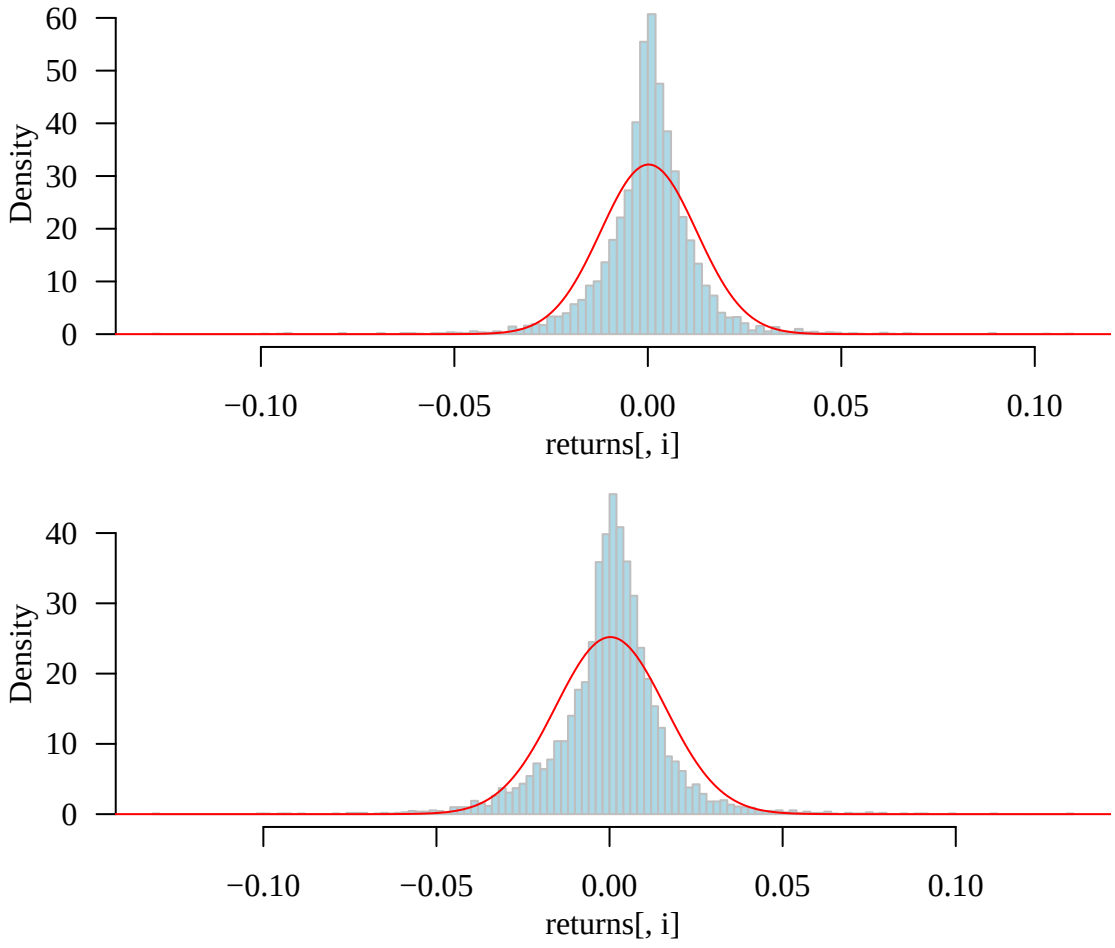

```
## Display parameter estimates and model-fit information
summary(copula_fit)
```

```
## Family
## ------
## No:    2
## Name:  t
##
## Parameter(s)
## ------------
## par:  0.92
## par2: 3.45
## Dependence measures
## -------------------
## Kendall's tau:    0.74 (empirical = 0.74, p value < 0.01)
## Upper TD:         0.68
## Lower TD:         0.68
##
## Fit statistics
## --------------
## logLik:  5228.59
## AIC:    -10453.19
## BIC:    -10439.95
```

Taking care of marginal distributions

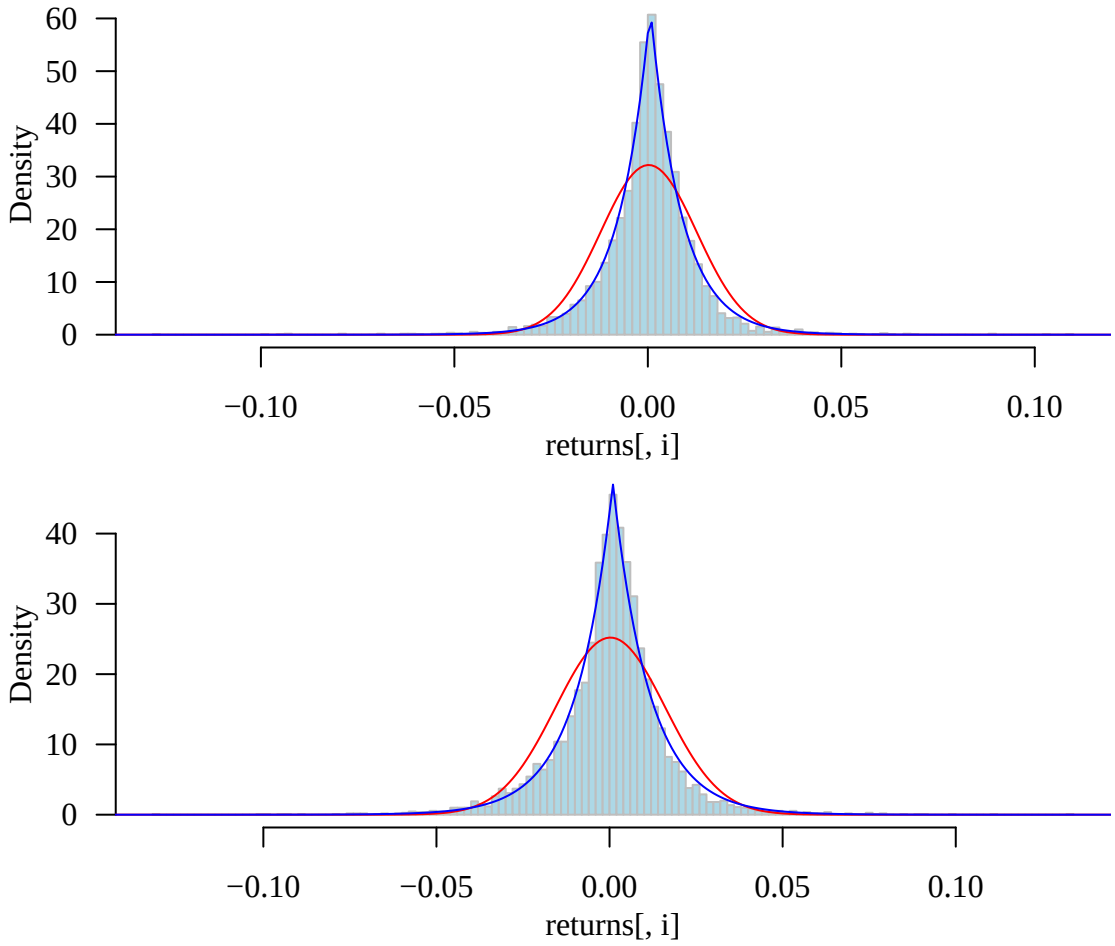
```
## Model the marginal distributions of the two return series

## Fit Normal distributions to each marginal return series
library(fitdistrplus)
```

```r
# Estimate the Normal mean and standard deviation for each index return
norm_fit <- apply(returns, 2, fitdist, "norm")

# Compare the empirical distributions with the fitted Normal densities
par(mfrow = c(2, 1), mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0),
    las = 1, family = "serif")

for (i in 1:2){
  # Plot the empirical distribution of returns
  hist(returns[, i], 100, col = "lightblue", border = "gray", prob = T,
       main = "")

  # Overlay the fitted Normal density
  lines(seq(-0.2, 0.2, 0.001), dnorm(seq(-0.2, 0.2, 0.001),
                                     norm_fit[[i]]$estimate[1],
                                     norm_fit[[i]]$estimate[2]),
        col = "red")
}
```



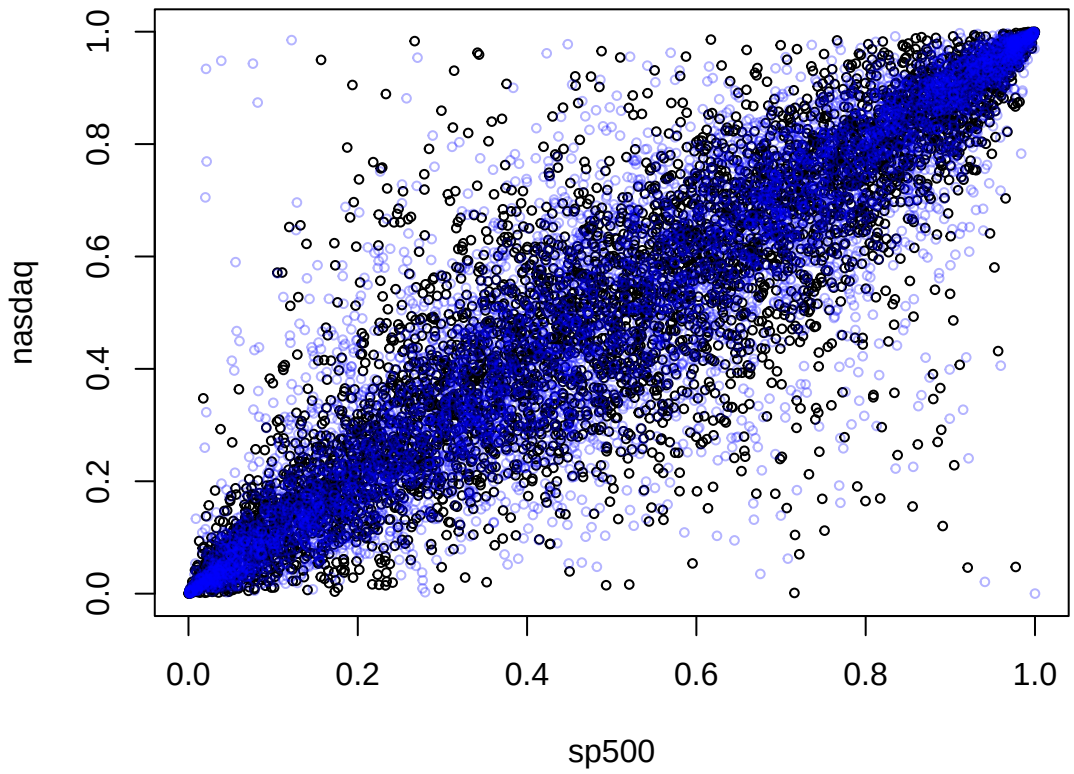

```r
## Fit Laplace distributions to each marginal return series

# Estimate the Laplace location by the sample median and
# the scale by the mean absolute deviation from the median
```

```r
Laplace_est <- apply(returns, 2, function(x) c(median(x), mean(abs(x - median(x)))))

library(rmutil)

# Compare the fitted Normal and Laplace densities
for (i in 1:2){
  # Plot the empirical distribution of returns
  hist(returns[, i], 100, col = "lightblue", border = "gray", prob = T,
       main = "")

  # Overlay the fitted Normal density
  lines(seq(-0.2, 0.2, 0.001), dnorm(seq(-0.2, 0.2, 0.001),
                                     norm_fit[[i]]$estimate[1],
                                     norm_fit[[i]]$estimate[2]),
        col = "red")

  # Overlay the fitted Laplace density for comparison
  lines(seq(-0.2, 0.2, 0.001), dlaplace(seq(-0.2, 0.2, 0.001),
                                        Laplace_est[1, i], Laplace_est[2, i]),
        col = "blue")
}
```



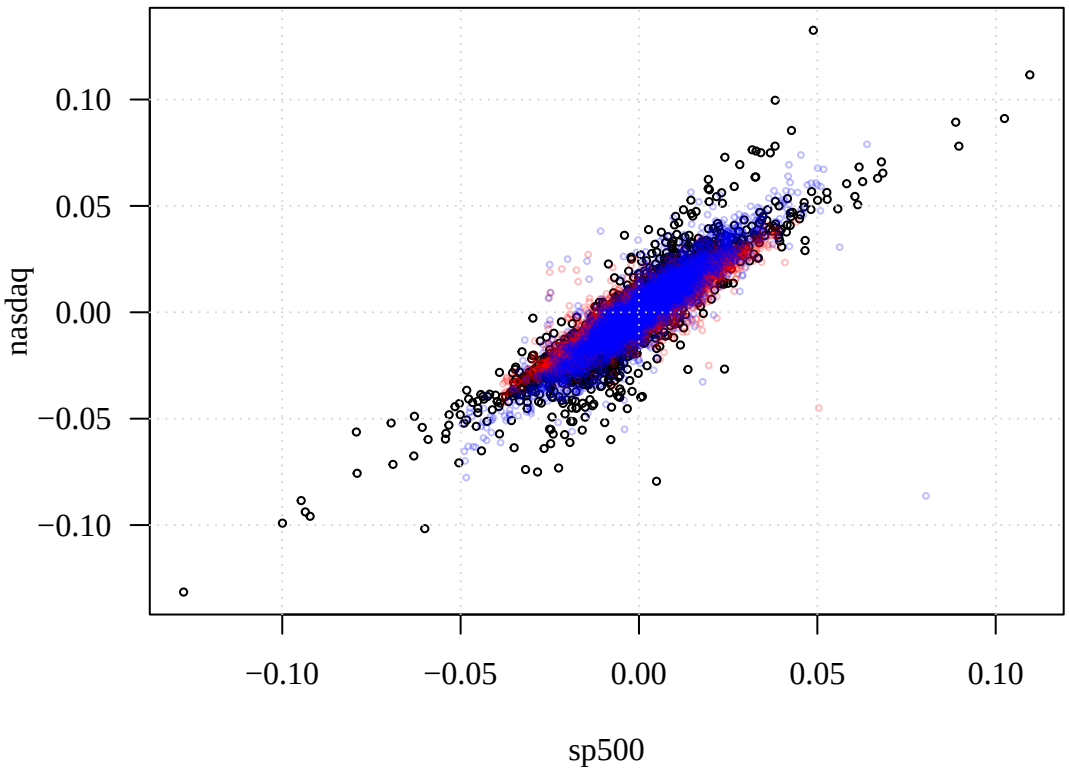
Let's put everything together to simulate new 'data'

```r
## Simulate observations from the fitted copula model

library(scales)

# Use the same sample size as the observed return data
n <- dim(returns)[1]

# Simulate new pseudo-observations from the selected copula
# These observations have Uniform(0,1) margins and the fitted dependence structure
u_sim <- BiCopSim(n, selectedCopula)

# Compare the observed pseudo-observations with those simulated from the fitted copula
plot(u, cex = 0.6)
points(u_sim, cex = 0.6, col = alpha("blue", 0.3))
```




```r
## Transform the simulated copula observations back to the original return scale

# Use the fitted Normal marginal distributions
x_sim1 <- qnorm(u_sim[, 1], norm_fit[[1]]$estimate[1], norm_fit[[1]]$estimate[2])
y_sim1 <- qnorm(u_sim[, 2], norm_fit[[1]]$estimate[1], norm_fit[[1]]$estimate[2])

# Use the fitted Laplace marginal distributions
x_sim2 <- qlaplace(u_sim[, 1], Laplace_est[1, 1], Laplace_est[2, 1])
y_sim2 <- qlaplace(u_sim[, 2], Laplace_est[1, 2], Laplace_est[2, 2])

## Compare simulated and observed returns

# Plot the observed pairs of S&P 500 and NASDAQ returns
```

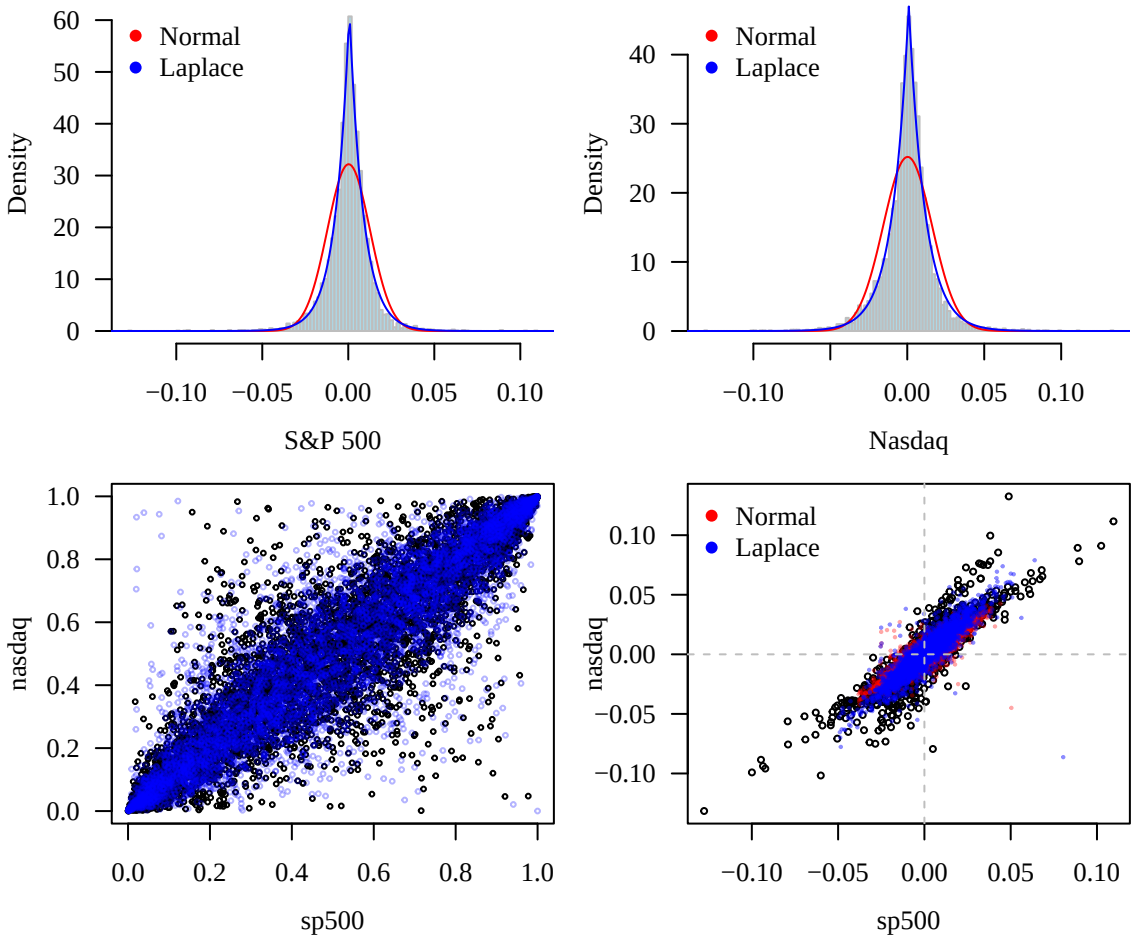
```r
par(las = 1, family = "serif")
plot(returns, cex = 0.5, las = 1)

# Overlay simulated returns using Normal margins
points(x_sim1, y_sim1, cex = 0.4, col = alpha("red", 0.25))

# Overlay simulated returns using Laplace margins
points(x_sim2, y_sim2, cex = 0.4, col = alpha("blue", 0.25))

grid()
```



The fitted copula describes the dependence between the two return series on the uniform probability scale. We can therefore simulate from the fitted copula and then use inverse marginal CDFs to transform the simulated values back to the return scale. Here we compare two choices of marginal models, Normal and Laplace, while keeping the same fitted copula dependence structure. This comparison illustrates an important advantage of copula modeling: the marginal distributions and the dependence structure can be modeled separately.

```r
## Compare the fitted marginal distributions and copula model
# Arrange the results in a 2 x 2 display
par(las = 1, mar = c(3.6, 3.6, 0.8, 0.6), mgp = c(2.5, 1, 0), mfrow = c(2, 2),
    family = "serif")

## Compare Normal and Laplace fits for each marginal return distribution
stock <- c("S&P 500", "Nasdaq")

for (i in 1:2){
  # Plot the empirical distribution of returns
  hist(returns[, i], 100, col = "lightblue", border = "gray", prob = T,
       main = "", xlab = stock[i])
```

```r
  # Identify the two fitted marginal models
  legend("topleft", legend = c("Normal", "Laplace"), pch = 16,
         col = c("red", "blue"), bty = "n")

  # Overlay the fitted Normal density
  lines(seq(-0.2, 0.2, 0.001), dnorm(seq(-0.2, 0.2, 0.001),
                                     norm_fit[[i]]$estimate[1],
                                     norm_fit[[i]]$estimate[2]), col = "red")

  # Overlay the fitted Laplace density
  lines(seq(-0.2, 0.2, 0.001), dlaplace(seq(-0.2, 0.2, 0.001),
                                        Laplace_est[1, i], Laplace_est[2, i]),
        col = "blue")
}

## Assess the fitted dependence structure on the copula scale
# Plot the observed pseudo-observations
plot(u, cex = 0.4)

# Overlay pseudo-observations simulated from the fitted copula
points(u_sim, cex = 0.4, col = alpha("blue", 0.3))

## Assess the complete joint models on the original return scale
# Plot the observed S&P 500 and NASDAQ return pairs
plot(returns, cex = 0.5)

# Overlay simulated returns using the fitted copula with Normal margins
points(x_sim1, y_sim1, cex = 0.2, col = alpha("red", 0.35))

# Overlay simulated returns using the fitted copula with Laplace margins
points(x_sim2, y_sim2, cex = 0.2, col = alpha("blue", 0.45))

# Add zero-return reference lines
abline(v = 0, lty = 2, col = "gray")
abline(h = 0, lty = 2, col = "gray")

# Identify the marginal models used for the simulated observations
legend("topleft", legend = c("Normal", "Laplace"), pch = 16,
       col = c("red", "blue"), bty = "n")
```

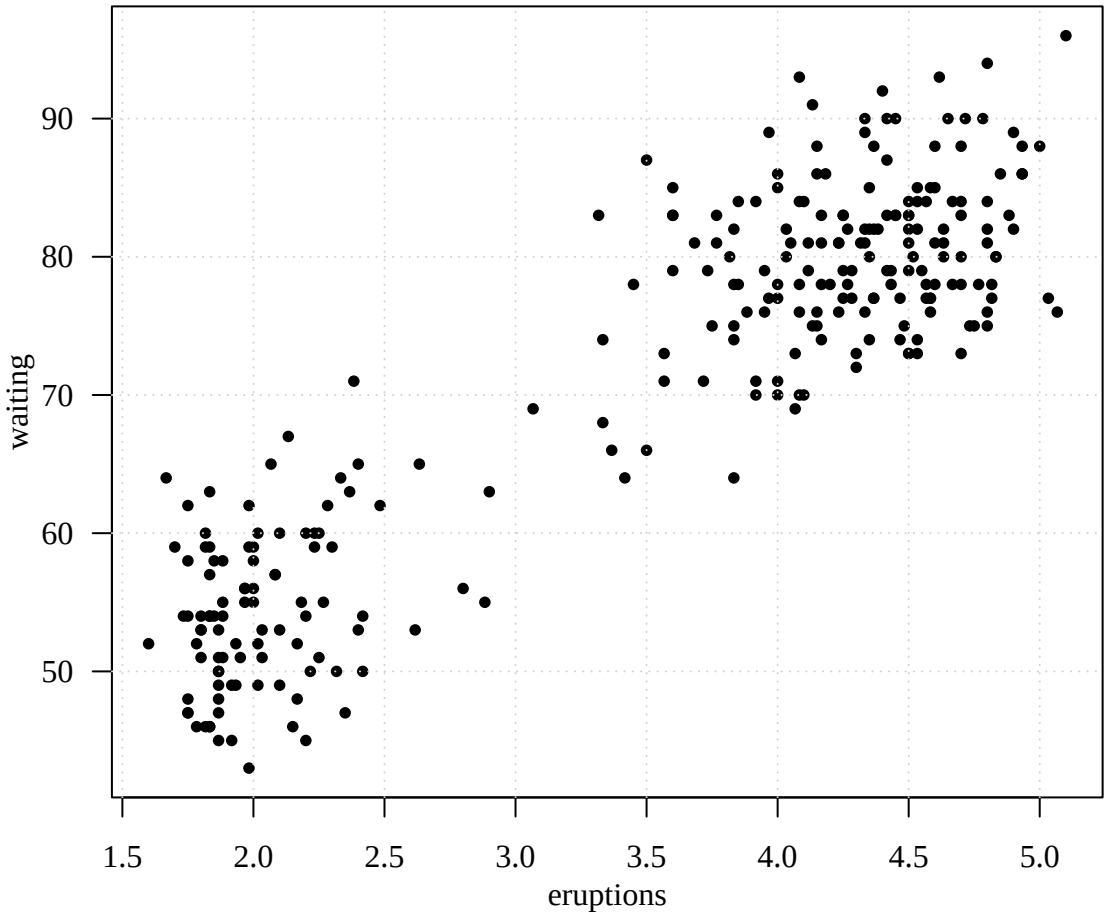


Nonparametric Density Estimation

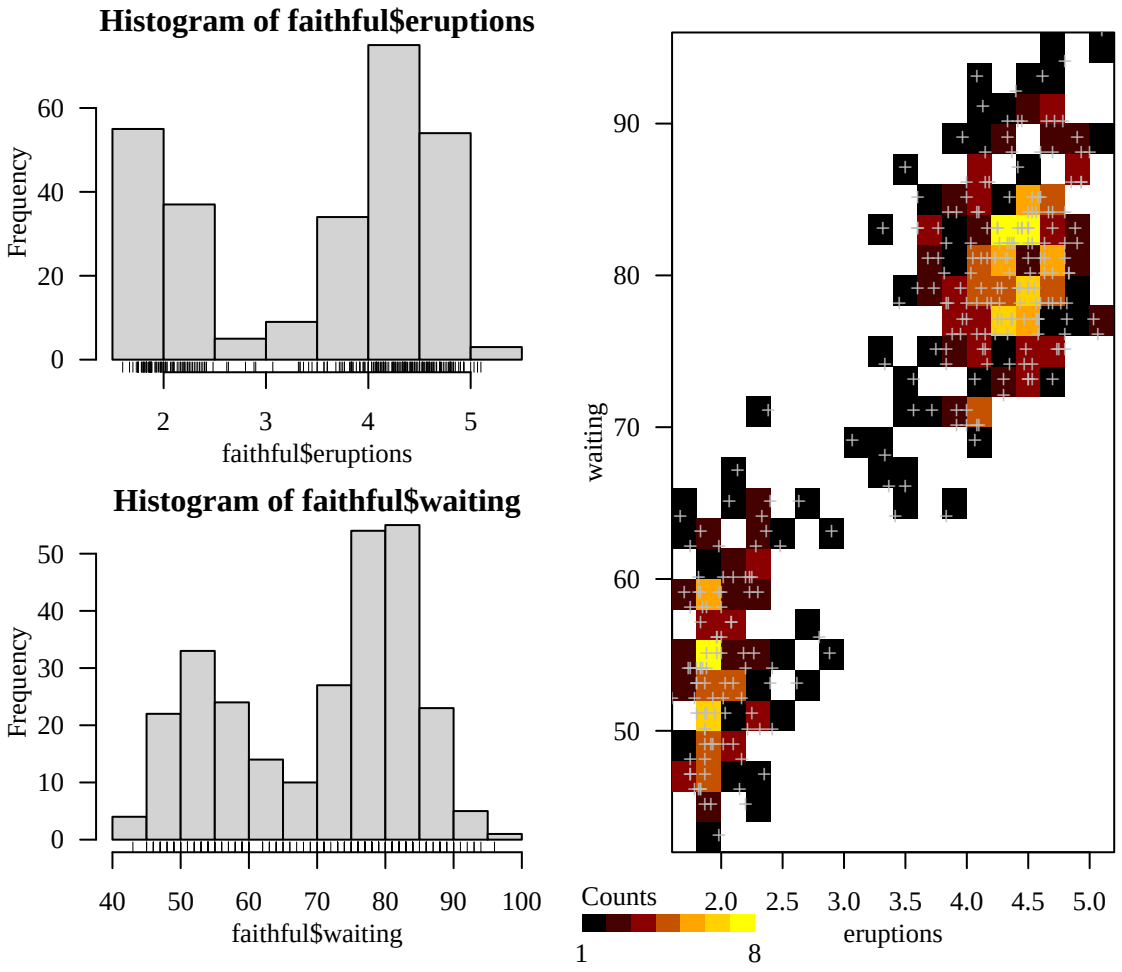
The `faithful` data contain observations from the Old Faithful geyser in Yellowstone National Park, recording eruption duration (`eruptions`) and the waiting time until the next eruption (`waiting`). We first explore the data graphically. The plots reveal clear structure, including bimodality, suggesting that a simple parametric model such as a single bivariate normal distribution may be too restrictive. This motivates more flexible approaches to estimating the underlying distribution.

Histogram

```
data(faithful)
## Explore the joint distribution of the Old Faithful data

# Plot eruption duration against waiting time to examine their joint structure
par(las = 1, mgp = c(2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6), family = "serif")
plot(faithful, las = 1, cex = 0.8, pch = 16)
grid()
```




```
## Examine both marginal and joint distributions

# Arrange two marginal histograms above a larger bivariate histogram
par(las = 1, mgp = c(2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6))
layout(matrix(c(1, 2, 3, 3), nrow = 2, ncol = 2))

# Examine the marginal distribution of eruption duration
hist(faithful$eruptions)
rug(faithful$eruptions)

# Examine the marginal distribution of waiting time
hist(faithful$waiting, las = 1)
rug(faithful$waiting)

# Construct a two-dimensional histogram to visualize the joint distribution
library(squash)
hist2(faithful, nx = 20, ny = 20, las = 1)

# Overlay the individual observations on the two-dimensional histogram
points(faithful, pch = "+", col = "gray")
```

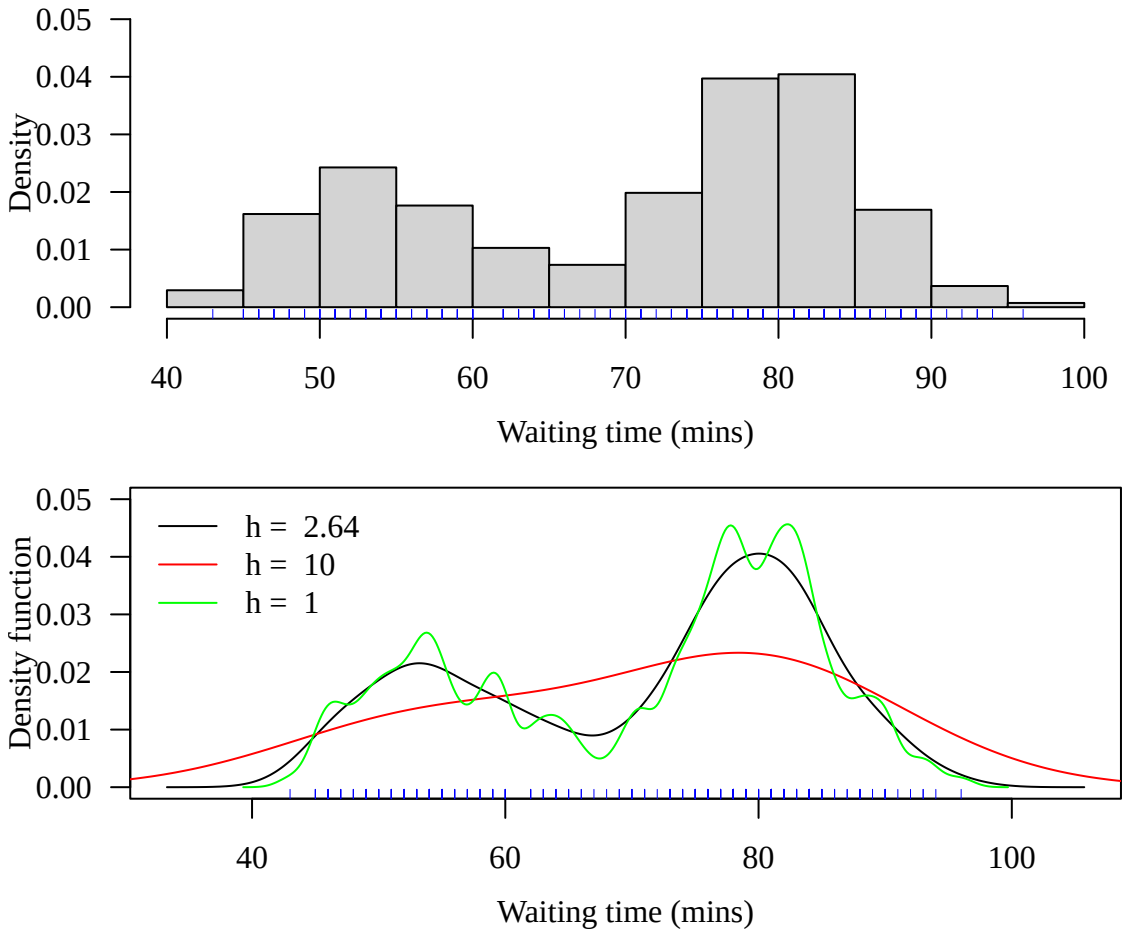


Transition from Histogram to Kernel Density

We now use kernel density estimation (KDE) to obtain a smooth estimate of the distribution of waiting times. Unlike a histogram, KDE does not require the specification of discrete bins. Instead, its appearance is primarily controlled by the bandwidth h, which determines the amount of smoothing. The following example compares the automatically selected bandwidth with a larger and a smaller bandwidth to illustrate the effect of this important tuning parameter.

```
## Estimate the density of Old Faithful waiting times using KDE
library(ks)

# Display the histogram and KDE comparison in two panels
par(las = 1, mfrow = c(2, 1), mar = c(3.6, 3.6, 0.8, 0.6), mgp = c(2.4, 1, 0),
    family = "serif")

## First examine the empirical distribution using a histogram
hist(faithful$waiting, xlab = "Waiting time (mins)", prob = T, main = "",
     ylim = c(0, 0.05))

# Mark the locations of the individual observations
rug(faithful$waiting, col = "blue")
```

```r
## Compare KDEs obtained using different bandwidths

# Plot the KDE using the automatically selected bandwidth
plot(kde(faithful$waiting), xlab = "Waiting time (mins)", ylim = c(0, 0.05))

# A larger bandwidth produces a smoother density estimate
plot(kde(faithful$waiting, h = 10), col = "red", add = T)

# A smaller bandwidth produces a less smooth, more variable density estimate
plot(kde(faithful$waiting, h = 1), col = "green", add = T)

# Identify the bandwidth used for each density estimate
legend("topleft", legend = paste("h = ", c(2.64, 10, 1)),
       col = c("black", "red", "green"), lty = 1, bty = "n")

# Mark the observed waiting times for reference
rug(faithful$waiting, col = "blue")
```

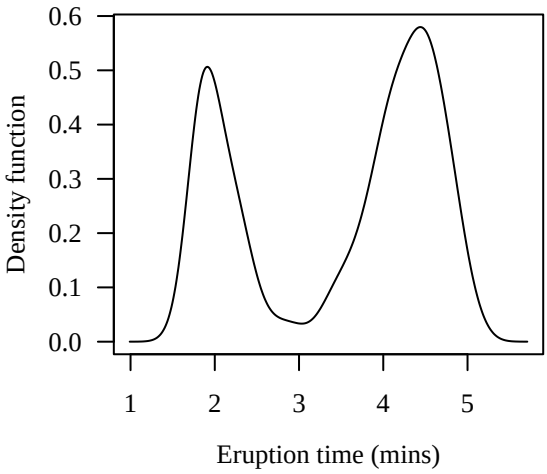


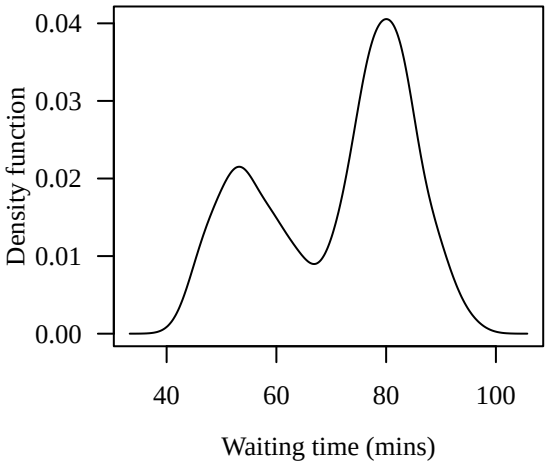
Kernel Density Estimation (KDE)

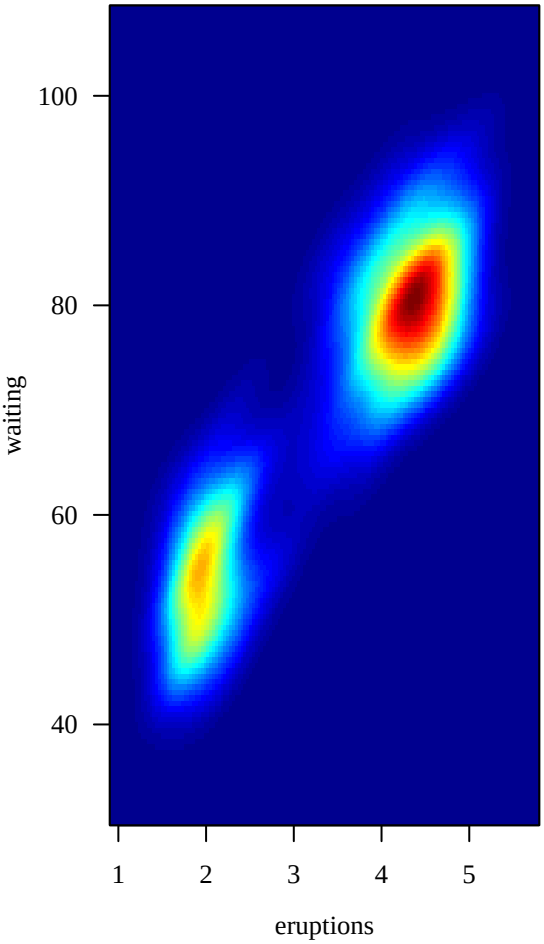
We now extend kernel density estimation from one dimension to the multivariate setting. We first estimate the marginal densities of eruption duration and waiting time separately. We then estimate their joint

bivariate density, where the scalar bandwidth h is replaced by a bandwidth matrix $\boldsymbol{H}$. Here, $\boldsymbol{H}$ is selected automatically using a plug-in bandwidth selector, allowing the amount and direction of smoothing to adapt to the joint structure of the data.

```
## Estimate the marginal and joint densities of the Old Faithful data

# Arrange the two marginal KDEs above the larger bivariate KDE
par(las = 1, mgp = c(2.6, 1, 0), mar = c(3.6, 4, 0.8, 0.6),
    family = "serif")
layout(matrix(c(1, 2, 3, 3), nrow = 2, ncol = 2))

## Estimate the two marginal densities separately

# KDE for eruption duration
plot(kde(faithful$eruptions), xlab = "Eruption time (mins)")

# KDE for waiting time
plot(kde(faithful$waiting), xlab = "Waiting time (mins)")

## Estimate the joint bivariate density

# Select a 2 x 2 bandwidth matrix H using the plug-in bandwidth selector
Hpi1 <- Hpi(x = faithful)

# Estimate the bivariate density using the selected bandwidth matrix
kdeHpi1 <- kde(x = faithful, H = Hpi1)

# Display the estimated joint density as a two-dimensional density image
plot(kdeHpi1, display = "image", col = tim.colors())
```


Density function
0.6
0.5
0.4
0.3
0.2
0.1
0.0
1
2
3
4
5
Eruption time (mins)


Density function
0.04
0.03
0.02
0.01
0.00
40
60
80
100
Waiting time (mins)


waiting
100
80
60
40
1
2
3
4
5
eruptions